SE601P

(Software construction and development Practical)

Objectives from Week 01 – 08

Prepared By: **ALI ABBAS**

BS Software Engineering

YouTube channel: **VU Facts**

https://www.youtube.com/@vufacts

SE601P – Week 1

Short Questions (3 Marks) – FINAL EXAM FORMAT

---

1. Define object-oriented software development.

Object-oriented software development is a programming approach that organizes software around objects. These objects represent real-world entities and contain both data and behavior. This approach improves modularity, reusability, and maintainability of software systems.

---

2. What is an object in object-oriented programming?

An object is an instance of a class that represents a real-world entity. It contains properties that define its state and methods that define its behavior. Each object occupies memory and can interact with other objects.

---

3. Define a class with an example.

A class is a blueprint or template used to create objects in object-oriented programming. It defines the properties and methods that objects of that class will have.
For example, a Student class may contain name and roll number as properties.

---

4. Differentiate between class and object.

A class is a logical blueprint that defines attributes and behaviors, while an object is a physical instance of a class. A class does not occupy memory, whereas an object occupies memory. Multiple objects can be created from a single class.

---

5. What are object properties? Give examples.

Object properties are attributes or characteristics that describe the state of an object. They store data related to the object.
For example, a car object may have properties such as color, model, and year.

6. What are methods in object-oriented programming?

Methods are functions defined inside a class that describe the behavior of an object. They perform actions on the object's data.
For example, start() and stop() are methods of a car object.

---

7. Define encapsulation.

Encapsulation is an object-oriented principle that combines data and methods into a single unit called a class. It restricts direct access to data from outside the class. This improves data security and program maintainability.

---

8. How does encapsulation improve software maintainability?

Encapsulation hides the internal implementation details of a class. Changes made inside the class do not affect other parts of the program. This makes the software easier to update and maintain.

---

9. What is information hiding?

Information hiding is a design principle that restricts access to internal details of a class or module. Only necessary information is exposed through public methods. This reduces dependency between different parts of a program.

---

10. Differentiate between encapsulation and information hiding.

Encapsulation is the process of bundling data and methods into a single unit. Information hiding focuses on restricting access to internal details. Encapsulation is a mechanism that supports information hiding.

---

11. Define inheritance.

Inheritance is an object-oriented mechanism that allows a class to acquire properties and methods of another class. The existing class is called the superclass, and the new class is called the subclass. Inheritance promotes code reuse.

---

12. Why is inheritance used in object-oriented programming?

Inheritance is used to reuse existing code and reduce duplication. It helps in creating hierarchical relationships between classes. This improves maintainability and extensibility of software.

---

13. What is polymorphism?

Polymorphism is an object-oriented concept that allows the same method to perform different actions for different objects. It enables flexibility and dynamic behavior in programs. Polymorphism is commonly achieved through method overriding.

---

14. Give one example of polymorphism.

An example of polymorphism is a sound() method behaving differently for different objects. A Dog object produces a bark sound, while a Cat object produces a meow sound. The method name remains the same, but behavior differs.

---

15. Define abstraction.

Abstraction is the process of showing only essential features of an object while hiding unnecessary details. It helps in reducing complexity and improving system understanding. Abstraction focuses on what an object does, not how it does it.

---

16. How is abstraction achieved in object-oriented programming?

Abstraction is achieved using abstract classes and interfaces. These define method declarations without implementation. The actual implementation is provided by subclasses or implementing classes.

---

17. What is object lifetime?

Object lifetime refers to the duration for which an object exists in memory. It starts when the object is created and memory is allocated. It ends when the object is destroyed and memory is released.

---

18. When is an object created and destroyed?

An object is created when memory is allocated and its properties are initialized. It is destroyed when the allocated memory is released. This usually happens when the object is no longer needed or the program ends.

---

19. Define association.

Association is a relationship between two or more objects where they are connected but can exist independently. It allows objects to interact with each other. Association does not imply ownership.

---

20. What is a bidirectional association?

A bidirectional association allows both related objects to access each other. Each object maintains a reference to the other. This enables two-way communication between objects.

---

21. Give an example of association.

An example of association is the relationship between a Customer and an Order in an e-commerce system. A customer can place orders, and orders can exist independently. Both objects interact with each other.

---

22. Define composition.

Composition is a strong form of association where a child object cannot exist without the parent object. The child object’s lifetime depends on the parent. Composition represents strong ownership.

23. Give one example of composition.

A car and its engine have a composition relationship. The engine cannot exist without the car. If the car is destroyed, the engine is also destroyed.

---

24. Define aggregation.

Aggregation is a relationship where a parent object contains child objects that can exist independently. It represents weak ownership. Child objects are not destroyed with the parent.

---

25. Give one example of aggregation.

A school and its departments have an aggregation relationship. Departments can exist independently of the school. They may also be shared among multiple schools.

---

26. Write one key difference between aggregation and composition.

In aggregation, child objects can exist independently of the parent. In composition, child objects cannot exist without the parent. Composition represents strong ownership, while aggregation represents weak ownership.

---

27. Which relationship shows strong ownership?

Composition shows strong ownership. The child object’s lifetime is controlled by the parent object. The child cannot exist independently.

---

28. Which relationship shows weak ownership?

Aggregation shows weak ownership. Child objects can exist independently of the parent. The relationship is less restrictive.

---

29. Can aggregated objects exist independently? Explain.

Yes, aggregated objects can exist independently. Their lifecycle is not controlled by the parent object. They may continue to exist even if the parent object is destroyed.

---

30. Can composed objects exist independently? Explain.

No, composed objects cannot exist independently. Their lifecycle depends on the parent object. If the parent object is destroyed, the composed objects are also destroyed.

Long Questions with Answers (5 Marks)

1. Object-Oriented Software Development

Answer (5 marks):
Object-Oriented Software Development (OOSD) is a programming methodology that focuses on designing software around **objects**, which are instances of classes.

- **Class:** A blueprint that defines the properties (attributes) and behaviors (methods) of objects.
- **Object:** A specific instance of a class with its own unique values for properties.

Principles of OOSD:

1. **Encapsulation:** Bundling data and methods together; hides internal details and provides interfaces.
2. **Inheritance:** A class can inherit properties and behaviors from another class, promoting code reuse.
3. **Polymorphism:** Objects can take multiple forms and behave differently in different contexts.
4. **Abstraction:** Simplifies complex systems by focusing only on relevant details.

Example: A `Car` class may have properties like `color`, `model`, `year` and methods like `accelerate()` and `brake()`. A specific car like `myCar` is an object of this class.

---

2. Object Properties and Methods

Answer (5 marks):
Objects have **properties** and **methods**:

- **Properties:** Attributes that define the object, e.g., `make`, `model`, `color` of a car.
- **Methods:** Actions the object can perform, e.g., `start()`, `stop()`, `turn()`.

Object Lifetime:

- **Creation:** Memory is allocated, and properties are initialized.
- **Existence:** Object can be accessed and modified until destroyed.
- **Destruction:** Memory is released when object is no longer needed.

Example: A `Book` object has properties `title` and `author`, and methods `open()` and `close()`.

---

3. Information Hiding

Answer (5 marks):
Information hiding is a design principle to **limit access** to internal details of a class.

- Only essential interfaces or methods are exposed.
- Helps reduce complexity, improve security, and maintain code.

Example: In a `BankAccount` class, balance is private, but `deposit()` and `withdraw()` methods allow controlled access.

---

4. Encapsulation

Answer (5 marks):
Encapsulation is the process of **bundling data and methods** into a single unit (class) and hiding internal details.

- Provides a **public interface** for interacting with the object.
- Protects the object from **unauthorized access** or modification.

Example:

```
class Student {
  private int marks;
  public void setMarks(int m) { marks = m; }
  public int getMarks() { return marks; }
}
```

---

5. Association, Composition, Aggregation

Answer (5 marks):

1. **Association:**
   - A relationship between two objects.
   - Can be one-to-one, one-to-many, or many-to-many.
   - **Example:** Customer and Order – a customer can have multiple orders.
2. **Composition:**
   - Strong ownership; parts cannot exist independently of the main object.
   - **Example:** Car and Engine – engine cannot exist without the car.
3. **Aggregation:**
   - Weak ownership; parts can exist independently.
   - **Example:** School and Department – departments exist even if the school object is deleted.

Week 2 – Short Questions (3 marks each)

1. What is information sharing between objects?
Answer:
Information sharing between objects is how objects communicate and exchange data using **method calls, parameter passing, or message passing** so that one object can use the information of another to perform actions.

2. What is generalization in OOP?
Answer:
Generalization is the process of creating a **new class (subclass)** from an existing class (superclass), inheriting its properties and methods while allowing additional features.

3. Define inheritance in object-oriented programming.
Answer:
Inheritance is a mechanism where a subclass acquires the **properties and methods of a superclass**, enabling code reuse and hierarchical class structures.

4. What is extension in OOP?
Answer:
Extension is adding **new properties or methods** to a subclass that are not present in its superclass.

5. What is method overriding?
Answer:
Overriding is redefining a **method inherited from the superclass** in the subclass to provide a specific implementation.

6. What is realization in OOP?
Answer:

Realization is when a **class implements an interface**, providing concrete definitions for all abstract methods defined in the interface.

7. Define polymorphism with an example.
Answer:
Polymorphism allows objects to take **multiple forms** depending on the context.
Example: A `Vehicle` object can act as `Car` or `Plane`, each having its own `move()` method.

8. List three ways objects share information.
Answer:

1. Method calls
2. Parameter passing
3. Message passing

Week 2 – Long Questions (5 marks each)

1. Explain information sharing between objects with examples.
Answer:
Information sharing allows objects to **communicate and use each other's data**.

- **Method Calls:** One object calls another object's method with parameters.
  - Example: `car.startEngine()` called by a `driver` object.
- **Parameter Passing:** Objects pass data or references directly.
  - Example: `order.placeOrder(customer)` passes the `customer` object.
- **Message Passing:** Objects send messages to each other to trigger actions.
  - Example: `email.send(message)` sends a notification from one object to another.

---

2. Explain generalization, inheritance, extension, and overriding with examples.
Answer:

- **Generalization:** Creating a subclass from a superclass to share properties.
  - Example: `Dog` class inherits from `Animal` class.
- **Inheritance:** Subclass acquires all methods and properties of superclass.
  - Example: `Dog` inherits `eat()` and `sleep()` from `Animal`.
- **Extension:** Adding new features to the subclass.
  - Example: `Dog` adds `bark()` method.
- **Overriding:** Redefining a method from superclass in subclass.
  - Example: `Dog` overrides `eat()` to show "Dog is eating".

---

3. Explain realization in OOP with an example.
Answer:

- **Realization:** A class implements an **interface**, defining all its abstract methods.
- **Purpose:** Allows **modular, flexible, and reusable code** without sharing implementation details.
- **Example:**

```
public interface Drivable {
  void drive();
}
public class Car implements Drivable {
  public void drive() {
    System.out.println("Car is driving");
  }
}
```

---

4. Explain polymorphism in OOP with a use case.
Answer:
Polymorphism allows **objects to behave differently** depending on the context:

- **Method Overloading:** Same method name, different parameters.
- **Method Overriding:** Subclass changes superclass method behavior.
- **Interfaces:** Different classes implement same methods differently.

Use Case Example:

```
class Vehicle { void move() { System.out.println("Vehicle moves"); } }
class Car extends Vehicle { void move() { System.out.println("Car drives"); }
}
class Plane extends Vehicle { void move() { System.out.println("Plane
flies"); } }
```

- Objects: `Vehicle v1 = new Vehicle(); Vehicle v2 = new Car(); Vehicle v3 = new Plane();`
- Calls `move()` show different behaviors depending on the object type.

---

5. Differentiate method overloading and method overriding.
Answer:

Feature	Method Overloading	Method Overriding
Same Method Name	Yes	Yes
Parameters	Must differ	Must be same
Return Type	Can differ	Must be same (or compatible)
Inheritance Required	No	Yes

Feature	Method Overloading	Method Overriding
Purpose	Flexibility in same class	Change behavior in subclass

---

6. Explain the importance of polymorphism and generalization in software development.
Answer:

- **Polymorphism:** Allows flexible, modular code; multiple forms reduce complexity.
- **Generalization:** Promotes code reuse; reduces duplication; supports hierarchical structures.
- **Example:** `Vehicle` superclass and `Car`, `Plane` subclasses using polymorphism allows calling `move()` on any vehicle object with correct behavior.

Week 3 – Short Questions (3 Marks)

1. What is generic programming?
Answer:
Generic programming is a way to write **reusable code** that works with multiple data types without rewriting it for each type.
Example: A generic class `<T>` in Java can handle Integer, String, or any object type.

---

2. Give one example of a constant in programming.
Answer:
```
final int MAX_SIZE = 100;
```
A constant is a value that **does not change** during program execution.

---

3. Name two good programming practices.
Answer:

1. Use **descriptive variable names**.
2. Write **small, focused functions** that perform a single task.

---

4. What is object serialization?
Answer:
Serialization is converting an **object's state into a byte stream** so it can be stored, transmitted, or reconstructed later.

---

5. What is a distributed object?
Answer:
A distributed object is an object that **runs on different systems** in a network and communicates using **RPCs or messaging protocols**.

---

6. Give an example of a peer-to-peer network use case involving object serialization.
Answer:
File sharing: Peer A serializes a file into bytes, sends it to Peer B, who deserializes it to reconstruct the file in memory.

Week 3 – Long Questions (5 Marks)

1. Explain generic programming with an example.
Answer:

- **Definition:** Generic programming allows writing **flexible, reusable code** for multiple data types.
- **Purpose:** Avoids duplicating code for different types, simplifies maintenance.
- **Implementation:** Uses **type parameters** like `<T>` in Java.
- **Example:**

```
class Test<T> {
    T obj;
    Test(T obj) { this.obj = obj; }
    public T getObject() { return this.obj; }
}
Test<Integer> iObj = new Test<Integer>(15);
Test<String> sObj = new Test<String>("Hello");
```

This allows `Test` class to store integers, strings, or any type.

---

2. Explain constants and good programming practices.
Answer:

- **Constants:** Values that **do not change**, declared using `final` or `const`.
  - Example: `final int MAX_AGE = 100;`
  - Use uppercase with underscores: `MAX_SIZE`.
- **Good practices:**
  1. Use **meaningful variable names**.

2. Keep **functions/classes small and focused**.
3. Comment complex code for clarity.
4. Test and **refactor regularly**.
5. Follow coding conventions for consistency.

---

3. Explain object model case study of a university system.
Answer:

- **Objects:** `Student`, `Course`, `Enrollment`.
- **Relationships:**
  - `Student` has `name`, `ID`, list of courses.
  - `Course` has `ID`, `name`, list of students.
  - `Enrollment` links `Student` and `Course`.
- **Purpose:** Demonstrates OOP concepts like **encapsulation, association, and composition**.
- **Example:** Accessing a student's enrolled courses or retrieving all students in a course.

---

4. Explain distributed objects and their benefits.
Answer:

- **Definition:** Objects that **run on multiple systems** in a network.
- **Communication:** Uses **RPCs, HTTP, SOAP, REST**.
- **Benefits:**
  1. Modular and scalable architecture.
  2. Fault-tolerant applications.
  3. Share resources across machines.
- **Technologies:** CORBA, Java RMI, .NET Remoting, Web Services.

---

5. Explain object serialization and deserialization with a peer-to-peer use case.
Answer:

- **Serialization:** Converts an object into a **byte stream** to transmit or store.
- **Deserialization:** Converts the byte stream back into the object in memory.
- **Use Case – P2P file sharing:**
  1. Peer A serializes a file and sends it to Peer B.
  2. Peer B deserializes the file into memory.
  3. File can be saved or used locally.
- **Importance:** Preserves object's state and metadata, ensures efficient and secure sharing.

Week 4 – Short Questions (3 Marks)

1. What is exception handling?
Answer:
Exception handling is a programming mechanism to **detect, manage, and respond** to unexpected errors during program execution without crashing the program.

---

2. Name the three main blocks used in exception handling.
Answer:

1. **try:** Contains code that may throw an exception.
2. **catch:** Handles the exception.
3. **finally:** Executes code regardless of whether an exception occurred.

---

3. Give an example of a system exception.
Answer:
`FileNotFoundException` – occurs when a program tries to open a file that does not exist.

---

4. Give an example of a user-defined (business) exception.
Answer:
`NegativeNumberException` – thrown when dividing by a negative number in a calculator program.

---

5. What is the difference between exception and error handling?
Answer:

- **Exception Handling:** Handles **recoverable errors** like division by zero or file not found.
- **Error Handling:** Handles **serious system errors** like `OutOfMemoryError`.

---

6. Which keyword is used to throw an exception in Java?
Answer:
The `throw` keyword is used to **manually throw an exception** from a method or block.

---

7. What is the purpose of the `throws` keyword?
Answer:
It declares that a method **may throw a certain type of exception**, so calling code can handle it.

Week 4 – Long Questions (5 Marks)

1. Explain exception handling with an example.
Answer:

- **Definition:** Exception handling manages unexpected errors to prevent program crashes.
- **Blocks:**
  1. `try` – code that may throw an exception.
  2. `catch` – handles the exception.
  3. `finally` – executes code regardless of exception.
- **Example:**

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("An error occurred: " + e.getMessage());
} finally {
    System.out.println("This code will always run.");
}
```

- **Explanation:** Dividing by zero throws `ArithmeticException`. Catch block handles it, finally block executes always.

---

2. Explain system and business exceptions with examples.
Answer:

- **System Exception:** Predefined errors caused by the environment.
  - Examples: `FileNotFoundException`, `IOException`, `NullPointerException`.
  - Handled using `try-catch`.

```
try {
    FileReader file = new FileReader("file.txt");
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
}
```

- **Business Exception:** User-defined errors for application logic.
  - Example: Insufficient balance in bank account.

```
public class WithdrawalException extends Exception {
```

```
    public WithdrawalException(String msg) { super(msg); }
}
```

---

3. Explain user-defined exceptions with an example.
Answer:

- **Definition:** Custom exceptions created for specific program requirements.
- **Steps:**
    1. Create a class extending `Exception`.
    2. Throw exception using `throw`.
    3. Handle using `try-catch`.
- **Example:**

```
public class NegativeNumberException extends Exception {
    public NegativeNumberException(String msg) { super(msg); }
}
public class Calculator {
    public static int divide(int a, int b) throws NegativeNumberException {
        if(b < 0) throw new NegativeNumberException("Cannot divide by
negative");
        return a / b;
    }
}
```

---

4. How do you handle multiple system exceptions? Give an example.
Answer:

- Use **multiple catch blocks** to handle different exceptions in a single try block.
- **Example:**

```
try {
    FileReader file = new FileReader("file.txt");
    BufferedReader reader = new BufferedReader(file);
    String line = reader.readLine();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    System.out.println("I/O error: " + e.getMessage());
} finally {
    System.out.println("Execution completed.");
}
```

---

5. Explain the keywords `throw` and `throws` with examples.
Answer:

- **throw:** Used to **manually throw an exception** in a method.
- **throws:** Declares that a method **may throw an exception**.

- **Example:**

```
public static void checkAge(int age) throws Exception {
    if(age < 18) throw new Exception("Age must be 18+");
}
try {
    checkAge(16);
} catch(Exception e) {
    System.out.println(e.getMessage());
}
```

- Here, `throw` raises the exception, and `throws` declares it in the method signature.

Week 5 – Short Questions (3 Marks)

Q1. What is an Integrated Development Environment (IDE)?

Answer:
An Integrated Development Environment (IDE) is a software application that provides a complete environment for software development. It includes tools such as a code editor, compiler/interpreter, debugger, and build automation tools in one place. IDEs help developers write, test, and debug code efficiently.

Q2. Give two examples of IDEs.

Answer:
Two examples of IDEs are:

- Eclipse
- IntelliJ IDEA

Q3. What is Visual Studio Code?

Answer:
Visual Studio Code is a lightweight code editor developed by Microsoft. It supports multiple programming languages through extensions and provides features like syntax highlighting, debugging, and version control integration.

Q4. What is a Version Control System (VCS)?

Answer:
A Version Control System is a tool used to manage changes to source code over time. It helps developers track modifications, collaborate with team members, and revert to previous versions when needed.

Q5. Name any two Version Control Systems.

Answer:
Two examples of Version Control Systems are:

- Git
- SVN (Subversion)

Q6. What is Git Bash?

Answer:
Git Bash is a command-line tool that provides a Linux-like terminal on Windows. It allows users to run Git commands and manage repositories easily using command-line operations.

Q7. What is the purpose of build automation tools?

Answer:
Build automation tools are used to automate the process of compiling, testing, and packaging software applications. They help reduce manual effort and ensure consistent builds.

Q8. Give two examples of build automation tools.

Answer:
Two examples of build automation tools are:

- Maven
- Gradle

Q9. What is Continuous Integration (CI)?

Answer:
Continuous Integration is a software development practice in which developers frequently integrate code changes into a shared repository. Each integration is automatically built and tested to detect errors early.

Q10. What is Continuous Delivery (CD)?

Answer:
Continuous Delivery is a practice where software is automatically prepared and kept ready for deployment after successful testing. It ensures that the application can be released to production at any time.

Week 5 – Long Questions (5 Marks)

Q1. Explain Integrated Development Environments (IDEs) with examples.

Answer:
An Integrated Development Environment (IDE) is a software application that provides a complete set of tools required for software development. IDEs combine a source code editor, compiler or interpreter, debugger, and build tools into a single interface.

IDEs increase developer productivity by simplifying code writing, error detection, and debugging. They also provide features like auto-completion, syntax highlighting, and project management.

Examples of IDEs include Eclipse, IntelliJ IDEA, Visual Studio, and Android Studio.

Q2. Explain Version Control Systems and their importance in software development.

Answer:
Version Control Systems (VCS) are tools used to manage and track changes made to source code over time. They allow multiple developers to work on the same project simultaneously without conflicts.

VCS helps in maintaining a history of changes, rolling back to previous versions, and resolving code conflicts. It also improves collaboration and code quality.

Examples of Version Control Systems include Git, SVN, and Mercurial.

Q3. Describe build automation tools and their role in software development.

Answer:
Build automation tools are used to automate tasks such as compiling source code, running tests, and packaging applications. These tools reduce manual effort and minimize human errors during the build process.

They ensure consistency and reliability in software builds, especially in large projects. Build tools also integrate well with CI/CD pipelines.

Common build automation tools include Maven, Gradle, and Ant.

Q4. Explain Continuous Integration and Continuous Delivery (CI/CD).

Answer:
Continuous Integration (CI) is a development practice where developers regularly integrate their code into a shared repository. Each integration triggers automated builds and tests to detect issues early.

Continuous Delivery (CD) extends CI by automatically preparing the application for deployment after successful testing. It ensures that the software is always in a deployable state.

CI/CD improves software quality, reduces deployment risks, and speeds up the development lifecycle.

Q5. Explain Git and its role in version control.

Answer:
Git is a distributed Version Control System used to track changes in source code. It allows developers to create repositories, manage branches, and merge code efficiently.

Git helps teams collaborate by keeping a complete history of code changes and allowing easy rollback when needed. It is widely used in modern software development due to its speed and flexibility.

Week 6 – Short Questions (3 Marks)

Q1. What is code review?

Answer:
Code review is the process in which developers examine each other's source code to identify bugs, improve code quality, and ensure coding standards are followed. It helps in early detection of errors and improves maintainability.

Q2. Why is code review important in software development?

Answer:
Code review improves code quality, reduces technical debt, and helps detect bugs early. It also ensures consistency and better collaboration among team members.

Q3. Define code review tools.

Answer:
Code review tools are software applications that automate and support the code review process by allowing developers to comment on code, track issues, and collaborate efficiently.

Q4. Give two examples of code review tools.

Answer:
Two examples of code review tools are GitHub and GitLab. These tools provide built-in features for reviewing code and collaborating with team members.

Q5. What are bug tracking tools?

Answer:
Bug tracking tools are used to record, track, and manage bugs and defects in software projects. They help teams prioritize and resolve issues efficiently.

Q6. Write two features of bug tracking tools.

Answer:
Bug tracking tools provide issue reporting and status tracking. They also support bug prioritization and assignment to team members.

Q7. What is the purpose of software documentation tools?

Answer:
Software documentation tools are used to create, manage, and maintain technical documentation such as user manuals and API documents in a centralized platform.

Q8. Give two examples of software documentation tools.

Answer:
Examples of software documentation tools include Confluence and Read the Docs. These tools help teams collaborate and keep documentation up to date.

Q9. What are testing tools?

Answer:
Testing tools are software applications used to automate and manage the testing process of software systems to ensure quality, reliability, and correctness.

Q10. Write two benefits of testing tools.

Answer:
Testing tools reduce manual testing effort and help detect bugs early. They also improve accuracy and ensure better software quality.

Q11. What is Selenium used for?

Answer:
Selenium is used for automated testing of web applications. It helps test browser-based functionalities efficiently.

Q12. What is the role of Jira in software development?

Answer:
Jira is a bug tracking tool used to track issues, manage tasks, and monitor project progress in software development.

Week 6 – Long Questions (5 Marks)

Q1. Explain code review and its importance in software development.

Answer:
Code review is a process in which developers examine each other's source code to identify errors and improve quality. It plays an important role in software development.

Importance of code review:

- Helps identify bugs and errors early
- Improves code quality and readability
- Ensures coding standards are followed
- Reduces technical debt
- Encourages collaboration among team members

Q2. Discuss code review tools with examples.

Answer:
Code review tools help automate and streamline the code review process. They allow developers to review code changes, provide feedback, and track issues efficiently.

Examples:

- GitHub: Provides pull request reviews and collaboration features
- GitLab: Offers built-in code review and CI/CD integration
- Bitbucket: Supports code review and integrates with Jira

These tools improve teamwork and maintain code quality.

Q3. Explain bug tracking tools and their benefits.

Answer:
Bug tracking tools are used to track, manage, and resolve bugs in software projects through a centralized system.

Benefits of bug tracking tools:

- Early identification and resolution of bugs
- Issue prioritization and assignment
- Improved software quality and stability
- Better team coordination
- Reporting and analytics for issue trends

Q4. What are software documentation tools? Explain their importance.

Answer:
Software documentation tools help create and manage documentation such as technical manuals and API documents.

Importance:

- Provide centralized documentation
- Improve communication among stakeholders
- Keep documentation up to date
- Support collaboration and version control
- Reduce knowledge gaps in teams

Q5. Explain testing tools and their role in software quality.

Answer:
Testing tools automate the testing of software applications to ensure functionality, performance, and reliability.

Role of testing tools:

- Automate repetitive testing tasks
- Detect bugs early in development
- Improve test accuracy and coverage
- Reduce manual testing effort
- Ensure compliance with quality standards

Q6. Differentiate between code review tools and bug tracking tools.

Answer:

Code Review Tools:

- Used to review source code
- Improve code quality
- Examples: GitHub, GitLab

Bug Tracking Tools:

- Used to track bugs and defects
- Manage issue lifecycle
- Examples: Jira, Bugzilla

Both tools improve software quality but serve different purposes.

Week 7 – Short Questions (3 Marks)

1. **What is Jira?**
   Answer:
   Jira is a project management and issue tracking tool used in software development. It helps teams plan, track, and manage tasks, bugs, and project progress efficiently.
2. **Why is a Jira account required?**
   Answer:
   A Jira account is required to access Jira features such as creating projects, adding issues, assigning tasks, and monitoring project progress.
3. **What is a Jira project?**
   Answer:
   A Jira project is a container that holds all issues, tasks, epics, and timelines related to a specific software application or development activity.
4. **What is meant by an issue in Jira?**
   Answer:
   An issue in Jira represents a unit of work such as a task, bug, or user story that needs to be completed within a project.
5. **Why is project setup important in Jira?**
   Answer:
   Project setup is important because it defines workflows, issue types, permissions, and settings, ensuring smooth and organized project management.
6. **What is an Epic in Jira?**
   Answer:
   An Epic is a large body of work in Jira that can be divided into smaller tasks or child issues for better planning and tracking.

7. **What are child issues?**
   Answer:
   Child issues are smaller tasks or issues that are linked to an Epic and represent detailed work required to complete the Epic.
8. **Why are timelines used in Jira?**
   Answer:
   Timelines are used to define start and end dates of tasks and Epics, helping teams plan work and meet project deadlines.
9. **What is the purpose of the Roadmap in Jira?**
   Answer:
   The Roadmap provides a visual overview of Epics, issues, and timelines, helping teams monitor project progress and future plans.
10. **Why is progress monitoring important in Jira?**
    Answer:
    Progress monitoring helps teams track completed and pending work, identify delays, and ensure that the project stays on schedule.

Week 7 – Long Questions (5 Marks)

1. **Explain the steps involved in creating and managing a project in Jira.**
   Answer:
   Creating and managing a project in Jira involves several steps. First, a user creates a Jira account to access the platform. Then, a new project is created by selecting a project type and template. After that, the project is set up by defining workflows, issue types, and permissions. Issues such as tasks and bugs are added and assigned to team members. Finally, progress is monitored using boards and the Roadmap to ensure timely project completion.
2. **Describe the role of issues in Jira project management.**
   Answer:
   Issues play a central role in Jira project management. They represent individual units of work such as tasks, bugs, or user stories. Issues allow teams to assign responsibilities, set priorities, track progress, and record work details. By managing issues effectively, teams can organize their workload, improve collaboration, and ensure that project goals are achieved.
3. **Explain the importance of Epics and child issues in Jira.**
   Answer:
   Epics are used to represent large pieces of work or major features in a project. They help in high-level planning and organization. Child issues break down an Epic into smaller, manageable tasks, making work easier to assign and track. Together, Epics and child issues improve project visibility, planning accuracy, and progress tracking.
4. **How does Jira support progress monitoring and planning?**
   Answer:
   Jira supports progress monitoring and planning through features such as boards, reports, and the Roadmap. Boards show the current status of issues, while reports provide insights

into completed and pending work. The Roadmap offers a visual timeline of Epics and tasks, helping teams plan future work and monitor overall project progress.

5. **Explain how Jira integrates with source code management tools and its benefits.**
   Answer:
   Jira can be integrated with source code management tools like GitHub and Bitbucket. This integration allows developers to link code commits, branches, and pull requests directly with Jira issues. It improves traceability, enhances collaboration between developers and managers, and provides better visibility into development progress.

Week 8 – Short Questions (3 Marks)

1. What is the first step to create a new project in Asana?
Answer:
The first step is to **create a new project** by selecting the "New Project" option in Asana.

---

2. Who can assign tasks to team members in Asana?
Answer:
The **Project Manager** is responsible for assigning tasks to team members.

---

3. What is the purpose of timelines for tasks in Asana?
Answer:
Timelines are used to **set start and end dates** for tasks and track deadlines.

---

4. What view shows pending and completed tasks for a developer?
Answer:
The **Developer View** displays the tasks assigned to a developer and their completion status.

---

5. What is a Work Space in Asana?
Answer:
A Work Space is a **central area to organize projects and teams** in Asana, where projects, tasks, and team members are managed.

Week 8 – Long Questions (5 Marks)

1. Explain the steps to create a new project and add tasks in Asana.
Answer:

- **Step 1:** Create a new project by selecting “New Project”.
- **Step 2:** Set up your project with a name and description.
- **Step 3:** Add tasks by clicking “Add Task” and assigning them to team members.
- **Step 4:** Set timelines and deadlines for each task.
- **Step 5:** Monitor progress using the Project Manager Dashboard and Developer View.

---

2. Explain the purpose of Project Manager Dashboard and Developer View in Asana.
Answer:

- **Project Manager Dashboard:** Used to **monitor project progress, deadlines, and task completion**. It provides a high-level overview of all projects and team activities.
- **Developer View:** Allows developers to **see pending and completed tasks** assigned to them, track progress, and update task status.
- Both views help in **effective project management** and ensure tasks are completed on time.

---

3. What is a Work Space and Team in Asana? Explain their importance.
Answer:

- **Work Space:** A **central area to organize projects and tasks**, accessible to all team members.
- **Team:** A group of users working together on the projects within a Work Space.
- **Importance:** Work Spaces and Teams ensure **organization, collaboration, and clear visibility** of all tasks and responsibilities, making project management efficient.

---

4. How do you set timelines for tasks in Asana and why is it important?
Answer:

- **Setting Timelines:** While adding a task, you can set a **start date and due date** to define the schedule.
- **Importance:** Timelines **help track deadlines, plan work, and avoid delays**, ensuring projects are completed on time.

---

5. Describe the process of monitoring progress in Asana.
Answer:

- **Project Manager Dashboard:** Gives an overview of all tasks, showing completed, pending, and overdue tasks.
- **Developer View:** Shows tasks assigned to individual developers with their status.
- **Roadmap/Timeline View:** Allows visualization of deadlines and task sequences.
- **Purpose:** Monitoring ensures **timely completion of tasks, accountability, and helps in resource planning**.